Relatório Técnico: Implementação e Replicação de Pipeline CI/CD

Unidade Curricular: Tecnologias de Desenvolvimento Web (TDW) — Módulo 1A (2026/2027)

Autor: Carlos Teixeira

Repositório GitHub: https://github.com/CarlT09/tdw-mp1-carlos-teixeira

Repositório GitLab: https://gitlab.com/tdw-mp1-group/tdw-mp1-carlos-teixeira

URL de Produção (Cloudflare Workers): https://tdw-mp1.carlosteixeira05.workers.dev

1. O Desafio e o Objetivo do Trabalho

Neste trabalho, o objetivo principal não foi desenvolver código aplicacional de raiz, mas sim construir toda a infraestrutura de automação (CI/CD) para levar uma aplicação Next.js (TypeScript) integrada com o Contentful Headless CMS até à produção no Cloudflare Workers.

O foco esteve em aplicar boas práticas de desenvolvimento: usar branches para cada funcionalidade (*feature branches*), proteger a branch principal (`main`) para que nada entre sem passar por Pull Request, garantir que o código tem qualidade antes de ser compilado e replicar toda esta lógica numa segunda plataforma (o GitLab CI/CD), percebendo as diferenças reais entre ferramentas.

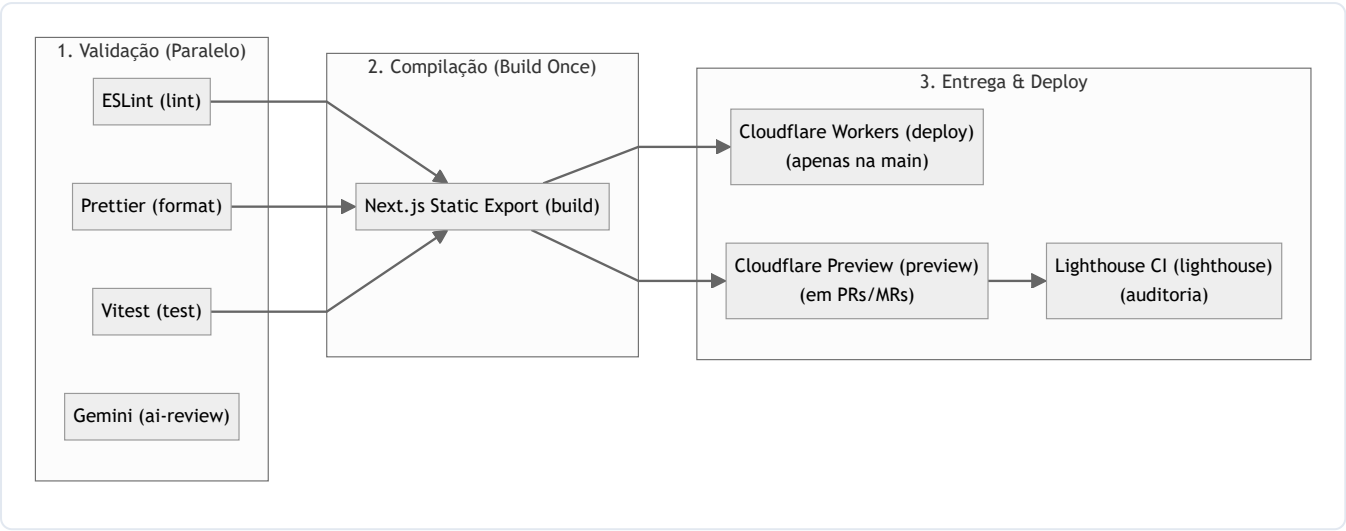
2. Escolha da 2ª Plataforma: GitLab CI/CD

Para replicar a pipeline que criei inicialmente no GitHub Actions, escolhi o **GitLab CI/CD** pelas seguintes razões:

- É uma das ferramentas mais usadas na indústria e no mercado de trabalho a nível de DevOps.
- Tem uma abordagem diferente do GitHub: enquanto o GitHub corre em máquinas virtuais completas recorrendo a ações prontas da comunidade, o GitLab organiza-se por estágios (*stages*) e corre diretamente em contentores Docker (`node:22`).
- Tem funcionalidades nativas muito interessantes, como os *Review Apps* (ambientes dinâmicos), o que permitiu comparar como as duas plataformas lidam com links de teste em Pull Requests / Merge Requests.

3. Como Estruturei a Pipeline de CI/CD

Em vez de correr todas as tarefas de forma sequencial (uma a seguir à outra), estruturei a pipeline para otimizar o tempo de execução e garantir que nenhum código com erros chega a produção.



3.1. Validações em Paralelo

Tanto no GitHub Actions (`.github/workflows/ci.yml`) como no GitLab (`.gitlab-ci.yml`), a pipeline começa por correr três verificações ao mesmo tempo:

- **ESLint (`lint`):** Confirma se não há erros de sintaxe ou de regras de código.
- **Prettier (`format`):** Confirma se o estilo e a formatação do código estão consistentes.
- **Vitest (`test`):** Corre os testes unitários.

O passo seguinte (`build`) só arranca se estes três trabalhos passarem com sucesso (através da instrução `needs: [lint, format, test]`).

3.2. Compilar uma só vez (*Build Once*)

Uma preocupação importante foi não perder tempo a compilar a aplicação várias vezes. No job `build` , o Next.js gera a versão estática do site para a pasta `./out` (`output: 'export'`). Esta pasta é guardada como um artefacto da pipeline.

Depois, os jobs que precisam desses ficheiros (tanto o `deploy` de produção como o `preview` de teste) limitam-se a descarregar essa mesma pasta e a enviá-la para a Cloudflare através da ferramenta `wrangler` . Isto garante que aquilo que foi testado é exatamente aquilo que vai para a nuvem.

3.3. Gestão de Segredos e Credenciais

Nenhum token ou chave secreta foi colocado no código:

- **Segredos:** Chaves de API (`CLOUDFLARE_API_TOKEN` , `CONTENTFUL_ACCESS_TOKEN` , `GEMINI_API_KEY`) foram configuradas em *Secrets* no GitHub e em variáveis mascaradas (*Masked*) no GitLab.
- **Identificadores normais:** Valores não secretos (`CLOUDFLARE_ACCOUNT_ID` , `CONTENTFUL_SPACE_ID`) ficaram como variáveis normais de repositório.
- No GitHub Actions, usei a sintaxe `${{ vars.VAR || secrets.VAR }}` para que a pipeline funcione quer o valor esteja guardado em variáveis ou em segredos.

3.4. Gatilhos da Automação

A pipeline reage a quatro situações:

- **Push na `main` :** Faz as validações, gera o build e coloca a nova versão em produção na Cloudflare.

- **Pull Request / Merge Request:** Cria uma versão de pré-visualização na Cloudflare com um URL único para podermos ver as alterações antes de fazer merge, corre o Lighthouse e chama o revisor de código IA.
- **Webhook do Contentful:** Quando crio ou edito um artigo no Contentful, o CMS dispara um webhook (`repository_dispatch`) para o GitHub, fazendo com que o site seja atualizado sem precisar de mexer no Git.
- **Agendamento diário (`schedule`):** Configurei um gatilho cron para correr à meia-noite (`0 0 * * *`). Depois de confirmar que funcionava, comentei esse gatilho no ficheiro YAML (com nota explicativa), tal como pedido no enunciado, para não gastar os minutos gratuitos da conta.

---

4. Extras e Bónus Implementados

Para além dos requisitos base, implementei várias melhorias práticas:

- **100% de Cobertura de Testes (Vitest):** Configurei a medição de cobertura com o `@vitest/coverage-v8`. Os testes cobrem 100% das linhas, funções e ramificações. Adicionei os badges no `README.md` do GitHub e configurei uma expressão regular no GitLab para mostrar a percentagem de 100% automaticamente na interface do Merge Request.
- **Testes de Snapshot:** Adicionei testes de snapshot com Vitest no ficheiro `constants.test.ts` para garantir que configurações fixas não mudam por acidente.
- **Auditoria com Lighthouse CI:** Tanto no GitHub como no GitLab, adicionei um job que audita o link de preview na Cloudflare, avaliando Performance, Acessibilidade e SEO antes do merge.
- **Revisor de Código com Inteligência Artificial (Google Gemini):** Criei um script próprio em Node.js (`scripts/ai-review.mjs`) que lê o *diff* do Pull Request/Merge Request e pede uma revisão técnica ao modelo `gemini-2.0-flash`. O script é independente de plataforma e corre da mesma forma no GitHub e no GitLab.

---

5. Comparação Prática: GitHub Actions vs. GitLab CI/CD

Ao replicar a pipeline no GitLab, notei diferenças evidentes no funcionamento de cada plataforma:

Aspeto	GitHub Actions	GitLab CI/CD
Estrutura	Ficheiro baseado em eventos (`on: push, pull_request`).	Ficheiro baseado em estágios (`stages: validate, build, deploy`).
Ambiente	Corre em máquinas virtuais Ubuntu completas (`runs-on: ubuntu-latest`).	Corre em contentores Docker limpos (`image: node:22`).
Cache de dependências	Fácil de ativar através do `cache: npm` no `setup-node`.	Definido manualmente indicando a pasta `.npm/` e a chave baseada no `package-lock.json`.
Links de Teste (Preview)	Tive de usar uma ação para comentar o link no PR.	O GitLab tem suporte nativo para **Review Apps**, colocando logo um botão "View app" no Merge Request.
Controlo de Pipelines	Cada job avalia os seus `if:` de forma independente.	Foi necessário usar `workflow: rules:` no topo para evitar que o GitLab criasse duas pipelines ao mesmo tempo (uma para a branch e outra para o MR).
Variáveis Protegidas	Os segredos ficam disponíveis em todas as branches por defeito.	No GitLab há o visto "Protect variable"; se ficar ativo, as variáveis só vão para a `main`, falhando nas branches de teste.

---

6. Problemas Reais que Tive e Como Resolvi

Durante o desenvolvimento deparei-me com alguns obstáculos práticos:

- **Conflito de dependências no Node 22 com o Vitest:** Ao correr `npm ci` dava erro de resolução de pacotes. Resolvi criando um ficheiro `.npmrc` com a opção `legacy-peer-deps=true` e regerei o `package-lock.json`, garantindo que o comando funciona sem falhas quer localmente quer na nuvem.
- **Configuração da proteção de branch no GitHub:** No início o merge ficava bloqueado porque tinha selecionado verificações com nomes manuais. Foi preciso alinhar as regras exatamente com os nomes dos jobs do GitHub Actions (`lint` , `format` , `test`).
- **Tempo de espera do link de preview na Cloudflare:** Quando o comando do Wrangler terminava, o URL às vezes demorava alguns segundos a responder na internet. Criei um ciclo de espera simples que faz pedidos até o site responder `HTTP 200` antes de arrancar com o Lighthouse.
- **Pipelines duplicadas no GitLab:** Quando abri o Merge Request no GitLab, a pipeline falhava a dizer que o job `preview` não encontrava o `build` . Isto acontecia porque o GitLab estava a gerar uma pipeline para a branch e outra para o MR ao mesmo tempo. Resolvi adicionando o bloco `workflow: rules:` que unificou o comportamento.
- **Ajuste na API do Gemini:** Inicialmente tive erros 404 porque o nome do modelo variava na Google AI Studio. Atualizei o script para interrogar a API sobre os modelos disponíveis na chave e aumentei o limite de tokens (`maxOutputTokens: 2500`) para a análise não ficar cortada a meio da frase.

---

7. O que Aprendi e Conclusão

Este trabalho permitiu-me perceber na prática como funciona uma pipeline profissional de CI/CD e porque é que estas ferramentas são tão importantes no dia a dia. Antes deste projeto, a ideia de automatizar testes, compilação e deploy parecia algo abstrato; ao implementar passo a passo, percebi o valor de ter a certeza de que nenhum código estragado vai para produção.

A experiência de replicar tudo no GitLab foi especialmente útil, porque mostrou que o importante não é decorar a sintaxe de uma ferramenta específica, mas sim entender os conceitos fundamentais: isolar tarefas, reutilizar artefactos, proteger credenciais e garantir que o processo é fiável e repetível em qualquer plataforma. Ambas as pipelines ficaram totalmente funcionais, verdes e com paridade completa.